

ОТЧЁТ ПО АУДИТУ БЕЗОПАСНОСТИ ВЕБ-ПРИЛОЖЕНИЯ

Лабораторная работа №7

Выполнил: Студент

Дата: 22.05.2026

Введение

Целью лабораторной работы №7 является проведение аудита безопасности веб-приложения, выявление уязвимостей и их устранение.

В отчете представлены:

1. Уязвимости, существовавшие до выполнения задания
2. Методы защиты, реализованные в процессе выполнения задания

1. ЗАЩИТА ОТ XSS (Cross-Site Scripting)

До исправления (уязвимость):

В коде отсутствовало экранирование при выводе пользовательских данных.

Уязвимый код в form.php:

```
<!-- Было: прямая вставка данных -->
<input type="text" name="FIO" value="<?= $values['FIO'] ?>">
<textarea name="bio"><?= $values['bio'] ?></textarea>
```

Риск: Злоумышленник мог ввести `<script>alert('XSS')</script>` в любое поле, и этот код выполнялся бы при просмотре страницы.

После исправления (защита реализована):

Исправленный код в form.php:

```
<!-- Стало: экранирование через htmlspecialchars -->
```

```
<input type="text" name="FIO" value="<?=
htmlspecialchars($values['FIO'] ?? ", ENT_QUOTES, 'UTF-8') ?>">
<textarea name="bio"><?= htmlspecialchars($values['bio'] ?? ",
ENT_QUOTES, 'UTF-8') ?></textarea>
```

Исправленный код в admin.php:

```
<!-- Вывод данных пользователя -->
<td><?= htmlspecialchars($user['name'], ENT_QUOTES, 'UTF-
8') ?></td>
<td><?= htmlspecialchars($user['email'], ENT_QUOTES, 'UTF-
8') ?></td>
```

Исправленный код в login.php:

```
<!-- Вывод сообщения об ошибке -->
<div class="error"><?= htmlspecialchars($_GET['error'] ?? ",
ENT_QUOTES, 'UTF-8') ?></div>
```

Результат: Все пользовательские данные экранируются при выводе.
XSS-атаки невозможны.

2. ЗАЩИТА ОТ INFORMATION DISCLOSURE

До исправления (уязвимость):

При ошибках SQL пользователь видел полный текст ошибки с путями к файлам.

Риск: Злоумышленник получал информацию о структуре БД, путях к файлам и версии PHP.

После исправления (защита реализована):

Исправленный код в db_config.php:

```
<?php
// Скрываем ошибки от пользователя
ini_set('display_errors', 0);
ini_set('display_startup_errors', 0);
error_reporting(E_ALL);

// Логируем ошибки в файл
ini_set('log_errors', 1);
```

```
ini_set('error_log', __DIR__ . '/logs/php_errors.log');

return [
    'host' => 'localhost',
    'dbname' => 'u82508',
    'user' => 'u82508',
    'pass' => '*****'
];
?>
```

Исправленный код в index.php:

```
try {
    // код работы с БД
} catch (PDOException $e) {
    error_log('Database error: ' . $e->getMessage());
    $messages[] = 'Произошла ошибка при загрузке данных.
    Пожалуйста, попробуйте позже.';
}
```

Исправленный код в login.php:

```
try {
    // код авторизации
} catch (PDOException $e) {
    error_log('Login error: ' . $e->getMessage());
    header('Location: login.php?error=Ошибка, попробуйте позже');
    exit();
}
```

Результат: Пользователь видит только общие сообщения об ошибках. Детали ошибок записываются в лог-файл.

3. ЗАЩИТА ОТ SQL INJECTION

До исправления (уязвимость):

Хотя в коде использовались подготовленные запросы, отсутствовала валидация типов для числовых параметров.

Уязвимый код в admin.php (было):

```
// Прямое использование без проверки типа
$request_id = $_GET['delete'];
```

```
$stmt = $db->prepare("DELETE FROM Frequest WHERE id = ?");  
$stmt->execute([$request_id]);
```

Риск: При передаче ?delete=1 OR 1=1 могло произойти нежелательное удаление записей.

После исправления (защита реализована):

Исправленный код в admin.php:

```
// Приведение к целому числу  
if (isset($_GET['delete']) && is_numeric($_GET['delete'])) {  
    $request_id = (int)$_GET['delete'];  
    $stmt = $db->prepare("DELETE FROM Frequest WHERE id = ?");  
    $stmt->execute([$request_id]);  
}
```

Исправленный код в index.php (подготовленный запрос):

```
$stmt = $db->prepare("SELECT request_id FROM UserInfo WHERE  
login = ?");  
$stmt->execute($_SESSION['login']);
```

Исправленный код в login.php (подготовленный запрос):

```
$stmt = $db->prepare("SELECT login, pass, request_id FROM UserInfo  
WHERE login = ?");  
$stmt->execute([$login]);
```

Валидация входных данных:

```
// Валидация email  
if (!filter_var($_POST['mail'], FILTER_VALIDATE_EMAIL)) {  
    // ошибка  
}
```

```
// Валидация телефона  
if (!preg_match('/^[+\d\s\-\(\)]{6,20}$/', $_POST['telep'])) {  
    // ошибка  
}
```

Результат: Все запросы используют подготовленные выражения PDO. Числовые параметры приводятся к типу int. SQL Injection невозможна.

4. ЗАЩИТА ОТ CSRF (Cross-Site Request Forgery)

До исправления (уязвимость):

В формах отсутствовали CSRF-токены. Злоумышленник мог отправить запрос от имени авторизованного пользователя.

Риск: Злоумышленник мог создать страницу с формой, отправляющей запрос на ваш сайт, и авторизованный пользователь отправил бы данные без своего ведома.

После исправления (защита реализована):

Создан файл csrf.php:

```
<?php
if (session_status() === PHP_SESSION_NONE) {
    session_start();
}

function generateCSRFToken() {
    if (empty($_SESSION['csrf_token'])) {
        $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
    }
    return $_SESSION['csrf_token'];
}

function verifyCSRFToken($token) {
    if (!isset($_SESSION['csrf_token']) || !isset($token)) {
        return false;
    }
    return hash_equals($_SESSION['csrf_token'], $token);
}

// Автоматическое обновление токена каждые 24 часа
if (!isset($_SESSION['csrf_time']) || $_SESSION['csrf_time'] < time() -
86400) {
    $_SESSION['csrf_time'] = time();
    $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
}
?>
```

Добавление токена в формы (form.php и login.php):

```
<form method="post" action="index.php">
  <input type="hidden" name="csrf_token" value="<?=
generateCSRFToken() ?>">
  <!-- остальные поля -->
</form>
```

Проверка токена в index.php:

```
if ($_SERVER['REQUEST_METHOD'] == 'POST') {
    if (!isset($_POST['csrf_token'])
|| !verifyCSRFToken($_POST['csrf_token'])) {
        die('Ошибка безопасности: CSRF атака обнаружена!');
    }
    // обработка данных
}
```

Проверка токена в login.php:

```
else {
    if (!isset($_POST['csrf_token'])
|| !verifyCSRFToken($_POST['csrf_token'])) {
        die('CSRF атака обнаружена!');
    }
    // обработка входа
}
```

Результат: Все формы защищены CSRF-токенами. Используется hash_equals() для защиты от timing-атак. Токены автоматически обновляются каждые 24 часа.

5. ЗАЩИТА ОТ INCLUDE УЯЗВИМОСТЕЙ

До исправления:

В приложении отсутствовал динамический include на основе пользовательского ввода. Угрозы не было.

После исправления (дополнительная защита):

Использование абсолютных путей:

```
require_once(__DIR__ . '/csrf.php');  
include(__DIR__ . '/form.php');
```

Результат: Все include используют фиксированные имена файлов и абсолютные пути. Уязвимость отсутствует.

6. ЗАЩИТА ОТ UPLOAD УЯЗВИМОСТЕЙ

Статус: Уязвимость неактуальна, так как в приложении отсутствует функционал загрузки файлов.

7. ДОПОЛНИТЕЛЬНЫЕ МЕРЫ БЕЗОПАСНОСТИ

7.1 Безопасные настройки сессии

Добавлено в index.php и login.php:

```
session_set_cookie_params([  
    'lifetime' => 3600,  
    'path' => '/',  
    'secure' => false,  
    'httponly' => true,  
    'samesite' => 'Strict'  
]);
```

7.2 Защита от брутфорса в login.php

Добавлено:

```
$max_attempts = 5;  
$lockout_time = 900; // 15 минут
```

```
if (!isset($_SESSION['login_attempts'])) {  
    $_SESSION['login_attempts'] = 0;  
    $_SESSION['last_attempt'] = time();  
}  
  
if ($_SESSION['login_attempts'] >= $max_attempts &&  
    (time() - $_SESSION['last_attempt']) < $lockout_time) {  
    die('Слишком много попыток входа. Попробуйте через 15 минут.');
```

7.3 Регенерация ID сессии после входа

Добавлено в login.php:

```
session_regenerate_id(true);
```